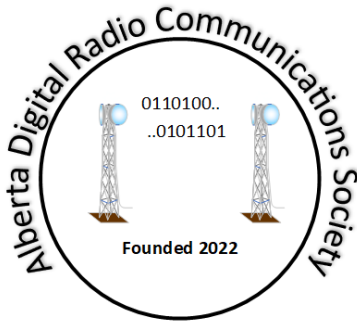




Digital Radio Projects

**IP400 Unified Firmware Platform
for Legacy WL33 based nodes**

Document Number: IP400-UFP-FSK

Revision: 0.5

Status: Released

Written By: Martin C. Alcock, M. Sc, MIEEE, VE6VH

Table of Contents

Revision Status	vi
Reference Documents.....	vi
Hardware Release Status	vi
Intellectual Property Notice.....	vii
Disclaimer.....	vii
Introduction	1
Roadmap	2
Hardware Platforms	3
Nucleo CC2 Experimenter Platform	3
Pi Zero HAT Mini-Node	10
Firmware Description.....	13
General Structure.....	13
Software Features	14
Subsystem detailed description	16
Host Interface.....	16
IP400 Frame Processor.....	24
Platform Support.....	30
Modems and Physical I/O	31
Operating the Software.....	33
Main Menu.....	33
Setup Parameters.....	34
Mesh Table	35
Chat/Echo Mode	36
Diagnostics Menu.....	38
Send Beacon.....	42
Radio Setup Menus	44
Station Setup Menu	45
Building the Code.....	46
Flashing	46
Support Software	47
UDP to SPI software	47
UDP Packet Generator.....	48

List of Tables

Table 1 Revision status	vi
Table 2 Reference Documents	vi
Table 3 Release Status.....	vi
Table 4 Supported platforms.....	3
Table 5 LPUART connections	4
Table 6 SPI connections from Nucleo to RPi	6
Table 7 Left hand Arduino Connector Pinouts	8
Table 8 Right hand Arduino Connector Pinouts	8
Table 9 Node MCU pin utilization.....	9
Table 10 Firmware Features and Module	14
Table 11 Firmware Features and Module	15
Table 12 Kiss framing characters	16
Table 13 KISS frame format.....	16
Table 14 Kiss Commands.....	17
Table 15 Test modes.....	18
Table 17 AX25 Frame Format.....	19
Table 18 AX.25 Address field format.....	19
Table 19 Address fields coding.....	19
Table 20 Command/Response bit coding	20
Table 21 SPI Frame Header format	21
Table 22 Frame flags definition	22
Table 23 Fragmentation field values	22
Table 24 Packet coding	22
Table 25 Modem qualifier fields	22
Table 26 SPI to IP400 Header Mapping.....	23
Table 27 Local Frame Processing.....	24
Table 28 Mesh Table Contents	25
Table 29 Header Symbol Coding	26
Table 30 Frame Header Flag field definitions.....	26
Table 31 Hop Table Flag field definitions	26
Table 32 Flag field values	27
Table 33 Station address coding.....	27
Table 34 Callsign character coding	28
Table 35 Radio Parameters.....	30
Table 36 Data mod values for OFDM-AB standards	30
Table 37 Data Symbols size definitions	30
Table 38 LED Usage by platform.....	32
Table 39 Main Menu Items	33
Table 40 Mesh Table explanation.....	35
Table 41 Capabilities field	35
Table 42 Chat mode control keys	36
Table 43 Diagnostics Menu	38

Table 44 Frame Statistics.....	39
Table 45 Frame Processor Statistics	40
Table 46 SPI Statistics	41
Table 47 Beacon Frame Payload.....	42
Table 48 LED Test Cycling	42
<i>Table 49 Setup parameters</i>	<i>44</i>
<i>Table 50 Permissible Bandwidth Values (KHz).....</i>	<i>44</i>
Table 51 Station setup parameters	45
Table 52 Code Components	46
Table 53 Pi utility software command parameters	47
Table 54 Pi Utility debug levels	47
Table 55 Loopback mode arguments	47
Table 56 Command line parameters for UDP packet generator	48
Table 57 Generated frame types.....	48

List of Figures

Figure 1 IP400 Node Development Roadmap.....	2
Figure 2 Nucleo CC2 Experimenter Platform	3
Figure 3 USB to UART Module examples	4
Figure 4 Adafruit GPS setup.....	5
Figure 5 Pi and Nucleo connections.....	6
Figure 6 Arduino Connector.....	7
Figure 7 Node MCU Pin Usage	9
Figure 8 Pi Zero HAT Mini-Node	10
Figure 9 IP400 System General Structure	13
Figure 10 Use of transposition in KISS frames	17
Figure 11 IP400 Frame Processor.....	24
Figure 12 Frame Fragmentation Schemes	29
Figure 13 4FSK Transceiver Stack	31
Figure 14 Main Menu.....	33
Figure 15 Setup Parameters.....	34
Figure 16 Mesh Table	35
Figure 17 Diagnostics Menu.....	38
Figure 18 WL33 Modem Statistics.....	39
Figure 19 Frame Processor Statistics.....	40
Figure 20 GPS NMEA sentences	43
<i>Figure 21 Radio Setup Menu</i>	<i>44</i>
Figure 22 Station Setup Menu	45

References

- [1] gnu.org, "General Public Licence," [Online]. Available: <https://www.gnu.org/licenses/gpl-3.0.en.html>. [Accessed 25th February 2018].
- [2] ST Microelectronics, "STM32WL33CC: Sub-GHz Wireless Microcontrollers.," ST Microelectronics, [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32wl33cc.html>. [Accessed 28 1 2025].
- [3] Digital Mobile Radio Association, "The DMR Standards," [Online]. Available: <https://www.dmrassociation.org/dmr-standards.html>. [Accessed 01 03 2026].
- [4] M. Chepponis and P. Karn, "The KISS TNC: A simple host-to-TNC communications protoocl," 1987. [Online]. Available: <https://www.ka9q.net/papers/kiss.html>. [Accessed 26 01 2026].
- [5] W. A. Beech, D. E. Nielsen and J. Taylor, "AX.25 Link Access Protocol," [Online]. Available: <https://ax25.net/AX25.2.2-Jul%2098-2.pdf>. [Accessed 25 01 2026].

Revision Status

Revision	Date	Description
0.1	February 2026	First Draft
0.2	March, 2026	Combined other docs
0.3	March, 2026	Reformatted around block diagrams
0.4	April, 2026	Added missing sections and merged in hardware docs
0.5	August, 2026	Revised as standalone document for legacy modems

Table 1 Revision status

Reference Documents

Author	Issue Date	Document Number	Description
M. Alcock	Jan 2025	IP400-PHY	Physical Layer Specification
M. Alcock	Mar 2025	IP400-SPI	Radio Node SPI Protocol specification
M. Alcock	Mar 2025	IP400-CC2	Nucleo-CC2 Experimenter Node

Table 2 Reference Documents

Hardware Release Status

Platform	Processor	Status	Base Version	Release Date
Nucleo CC2	STM32WL33	Available from Manufacturer	2.0	Q3, 2026
Mini Node	STM32WL33	No longer manufactured		

Table 3 Release Status

Intellectual Property Notice

The hardware components and all intellectual property described herein is the exclusive property of the Alberta Digital Radio Communications Society and others (“the owners”), all rights are reserved.

The owners grant licence to any Amateur for personal or club use, on an as is and where is basis under the condition that its use is for non-commercial activities only, all other usages are strictly prohibited. Terms and conditions are governed by the GNU public licence [1].

No warranty, either express or implied or transfer of rights is granted in this licence and the owner is not liable for any outcome whatsoever arising from such usage.

Copyright © Alberta Digital Radio Communications Society, all rights reserved. Not for publication.

Disclaimer

This document is a preliminary release for a product still in development and may be subject to change in future revisions. If any page in this document is blank, it is completely unintentional.

Introduction

The IP400 project was launched to experiment with digital mesh networking on the 400 MHz band, using commercial off the shelf devices. To get the project rolling, the initial firmware was developed for an STM32WL33 microcontroller [2], on several platforms that support the basic mode.

As the project has matured, new data modes have been explored, to satisfy the stated project goals. At the ZRDC conference in September of 2025, two of the stated goals were to implement a mode that could be retrofitted to existing repeater systems and off the shelf radios, complete with a mobile TNC terminal. Several modes were investigated, including a derivative of the Digital Mobile Radio (DMR) protocol [3], which led to developing the concept of an audio band variant of OFDM for this purpose, dubbed OFDM-AB.

However, the first nodes were not left behind. The UFP was coded so that the mesh networking component is separated from the modem component, so that all the features are the same regardless of the modem that is used. As of the current release, both the legacy 4FSK and OFDM-AB modes are supported. The firmware is compiled with the modem of choice by specifying a conditional compilation variable.

This document describes the UFP when compiled for the legacy modems that are based on the STM32WL33 processor, which is used in the Nucleo experimenter module, and the mini node. This modem has only one supported mode, 4FSK at 100KB/s, which conforms to the IEEE 802.15.4 standard is legal to operate in most countries.

Roadmap

Figure 1 illustrates the roadmap for the IP400 nodes and firmware development. This document describes the second phase of the project.

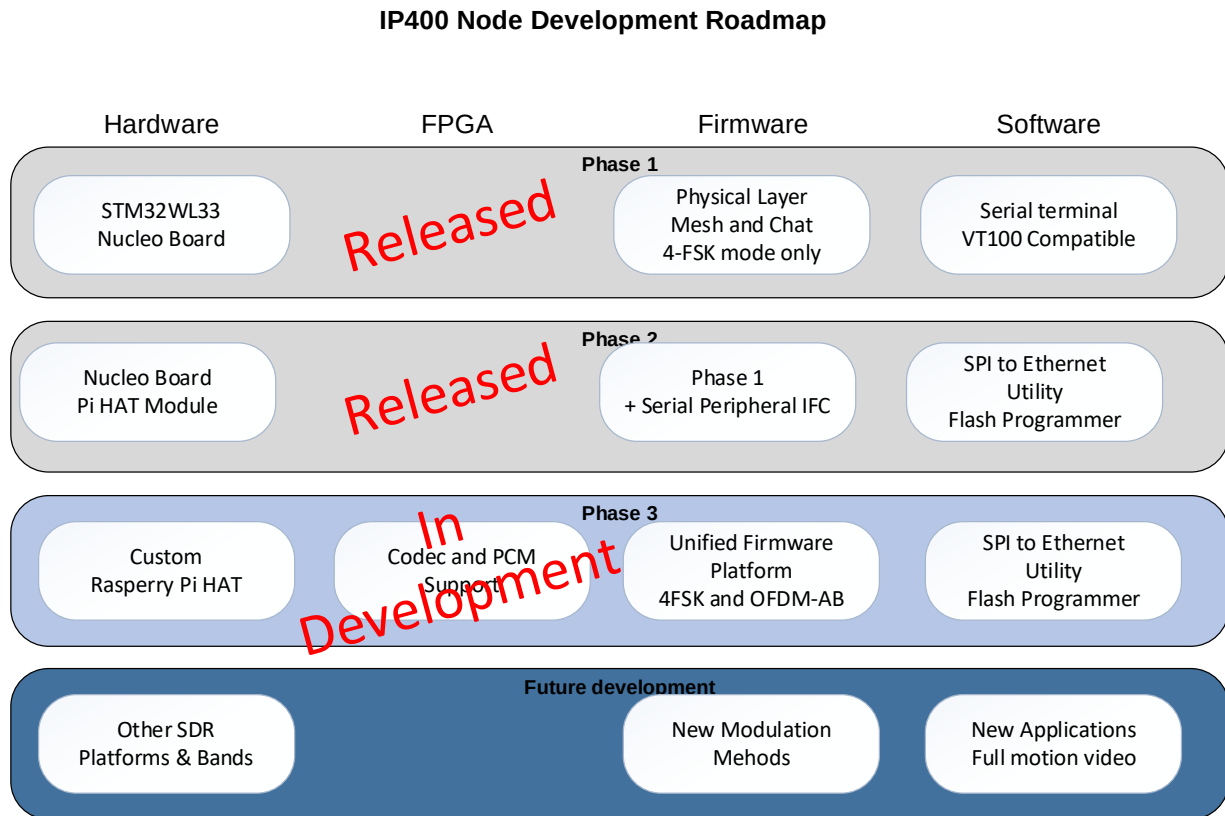


Figure 1 IP400 Node Development Roadmap

Hardware Platforms

Table 4 describes the various hardware platforms, radio modes and host type and interfaces.

Platform	Processor	Host and Interfaces	Mode A	Mode B	Mode C
Nucleo CC2	STM32WL33	Raspberry Pi SPI	Built in with Max +14dBm	n/a	n/a
Mini Node	STM32WL33	Pi Zero SPI1, UART	Built in with Max +20dBm	n/a	n/a

Table 4 Supported platforms

Nucleo CC2 Experimenter Platform

This module is designed for the experimenter, as it not only has the capability to develop code for the processor, but also has Arduino-style connectors that can be used as a platform for different types of experimentation, including adding a GPS receiver or other modules. A built-in debug capability enables downloading and software breakpoints to be inserted for debugging capabilities. It can also be connected to a raspberry Pi using the SPI connections, to take advantage of code developed for the host. The RF power output is limited to +14dBm. Figure 2 illustrates the module.



Figure 2 Nucleo CC2 Experimenter Platform

Using the Second UAR/T

The second UAR/T (also know as LPUAR/T, or Low Power UAR/T), can be utilized for a variety of applications, such as a GPS receiver, or to implement a simple modem connection. The Nucleo board has several connectors, including the Arduino style which can support GPS receivers or any type of prototyping board.

Signal	WL33 Pin	GPIO	Morpho CN4
LPUART_TX	PB6	54	35
LPUART_RX	PB7	55	37
GND	-	-	32

Table 5 LPUART connections

Implementing a USB Connection

An external USB connection to the second LPUART can be implemented with an external module such as those illustrated in Figure 3. Any evaluation board for a device from such manufacturers as Future Technology Devices International (FTDI), or Cypress can be utilized¹. The modules can be connected to the Nucleo board using the pins referenced in Table 5. Ensure that the Tx and Rx are crossed over correctly.



Figure 3 USB to UART Module examples

¹ The CP2102 as show has a maximum bit rate of 720Kb/s, for higher data rates the FT2232H is recommended

Connecting a GPS receiver

A GPS receiver may be connected to the Nucleo using the Arduino connectors. The recommended module is the 'Ultimate GPS logging shield' from Adafruit industries **Invalid source specified..** The connection requires a small board modification as shown in Figure 4.

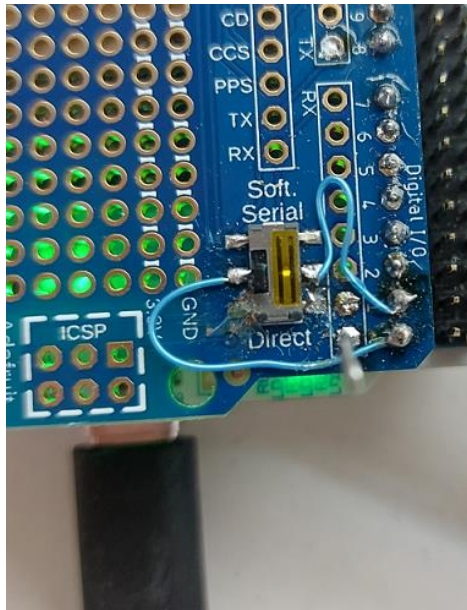


Figure 4 Adafruit GPS setup

The modification is as follows:

1. Using some soldering braid and an X-Acto knife, remove the solder from both pins at the lower left hand and right hand side of the slide switch, then lift the pin or remove it completely.
2. With an ohmmeter, check there is no continuity between the left pin on the switch and the I/O pin labelled '1', and the right pin and the I/O pin labelled '0'.
3. Solder a piece of #30 Kynar wire to the centre pin on the left side, then to the pin labelled '0' on the connector, and a second wire from the right hand side to the pin labelled '1' on the connector.
4. Check continuity with an ohmmeter between the slide switch and the new pin locations.
5. Ensure that the switch is in the 'Direct' (towards the removed pins) position.

Connecting the Nucleo to a Raspberry Pi

As the same code runs both on the Pi HAT as on the Nucleo board, a Nucleo board can be connected to a RPI to offer the same functionality as the HAT board, however the code has to be compiled and installed on the Nucleo using the IDE. Unzip and import the WL33_NUCLEO_UART project, add the latest code to the workspace, compile and download to the Pi.

The connection is made using cable jumpers from CN4 on the Nucleo board's morpho connector, to the HAT connector on the raspberry Pi. The table below lists the signal name, its origin on the WL33 chip, the GPIO designation, and the pin where it can be found on the morpho, plus the corresponding pin on the Raspberry PI HAT connector.

Wire Colour	Nucleo Signal	WL33 Pin	Morpho CN4	Pi Signal	GPIO	Pi Pin
White	SPI_MISO	PB8	15	SPIO_MOSI	10	19
Brown	SPI_MOSI	PB9	13	SPIO_MISO	9	21
Orange	SPI_SCK	PB11	11	SPIO_SCK	11	23
Yellow	SPI_NSS	PB10	17	SPIO_CEO ²	8	24
				SPIO_CE1	7	26
Green	GND	-	9	GND	-	25
	Test Point	PA6	3	-	-	-

Table 6 SPI connections from Nucleo to RPi

Figure 5 illustrates the Pi to Nucleo connections using jumpers.

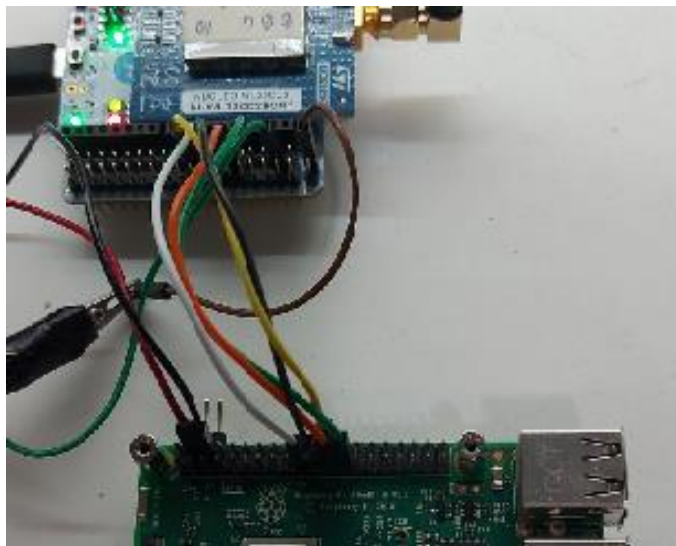


Figure 5 Pi and Nucleo connections

² Pick either CEO or CE1.

Test Point

A test point is provided at Pin 3 of the Morpho connector. This is toggled each time a data exchange occurs between the Pi and the Nucleo board. It can be connected to an oscilloscope and should appear as a square wave toggling every 20ms, or a 40 ms period.

The frequency of the toggling can be changed in the code, see the referenced document on the SPI specification.

Arduino Connectors

Figure 6 illustrates the Arduino connector on the Nucleo main board.

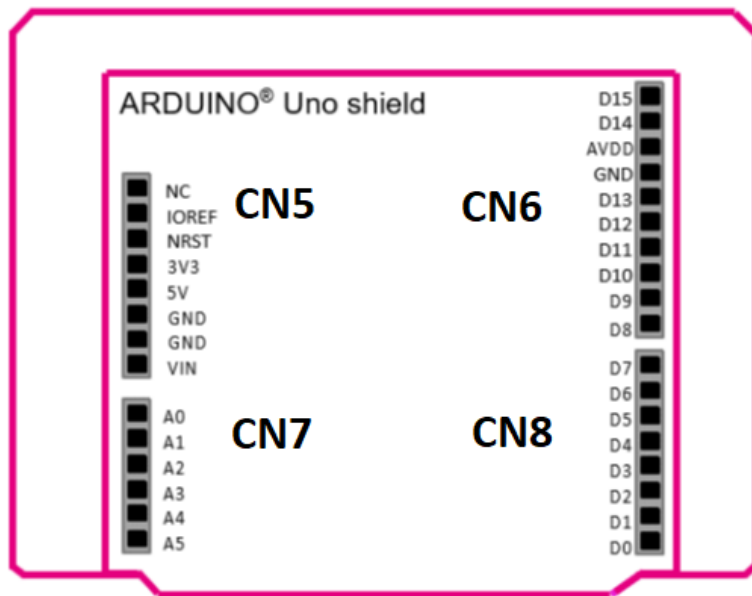


Figure 6 Arduino Connector

Left hand side connector pinouts (CN5 and CN7)

Connector	Pin Number	Pin Name	MCU Pin	Function
CN5	1	NC	-	
	2	3V3 (IOREF)	-	IOREF
	3	NRST	NRST	Low Active Reset
	4	3V3	-	3.3V output
	5	5V	-	5V
	6	GND	-	GND
	7	GND	-	GND
	8	VIN	-	Ext Supply 7-12V
CN7	1	A0	PB0	ADC_VINM1
	2	A1	PB1	ADC1_VINP1/LCA
	3	A2	PB2	ADC1_VINM0/LCB
	4	A3	PB3	ADC_VINP0
	5	A4	PB4	PVD_VIN
	6	A5	PB5	ADC_VINP3/LD3

Table 7 Left hand Arduino Connector Pinouts

Right hand side connectors (CN6 & CN8)

Connector	Pin Number	Pin Name	MCU Pin	Function
CN8	1	D0	PB7	LPUART1_RX
	2	D1	PB6	LPUART1_TX
	3	D2	PA9	USART1_TX
	4	D3	PA0	GPIO/B1
	5	D4	PA8	USART1_RX
	6	D5	PA13	GPIO
	6	D6	PA12	GPIO
	7	D7	PA4	GPIO
CN6	1	D8	PA5	GPIO
	2	D9	PA14	GPIO
	3	D10	PB10	SPI1_NSS
	4	D11	PB9	SPI1_MOSI
	5	D12	PB9	SPI1_MISO
	6	D13	PB11	SPI1_SCK
	7	GND		
	8	AVDD		
	9	D14	PA7	I2C2_SDA
	10	D15	PA6	I2C2_SCL

Table 8 Right hand Arduino Connector Pinouts

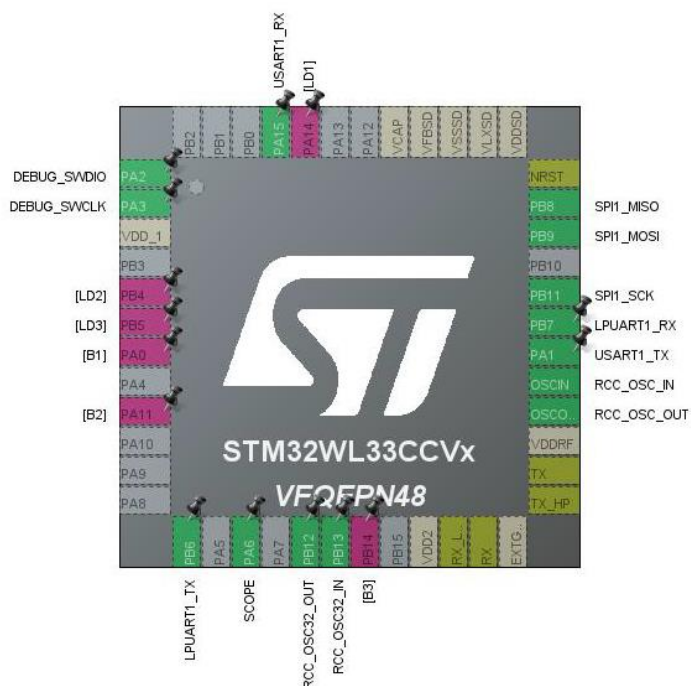


Table 9 lists the pins used by the node MCU application. The power and clock pins are not listed.

Pin	Name	Usage
PA2	SWDIO	Serial data used by debugger
PA3	SWCLK	Serial clock used by debugger
PA14	LD1	Blue LED
PB4	LD2	Green LED
PB5	LD3	Red LED
PA0	B1	Pushbutton 1
PA11	B2	Pushbutton 2
PB6	LPUART_TX	LPUART Tx data. Connected to Arduino CN8/D1
PA6	SCOPE	Scope trigger used by SPI module
PB14	B3	Pushbutton 3
PA1	USART1_TX	USART1 Tx. Connected to STLink VCOM port
PB7	LPUART_RX	LPUART Rx data. Connected to Arduino CN8/C0
PB11	SPI1_SCK	SPI Serial clock for Rpi connection
PB9	SPI1_MOSI	SPI Master out, slave in
PB8	SPI_MISO	SPI Master in, slave out
PA15	USART1_RX	USART1 Rx. Connected to STLink VCOM port

Table 9 Node MCU pin utilization

Pi Zero HAT Mini-Node

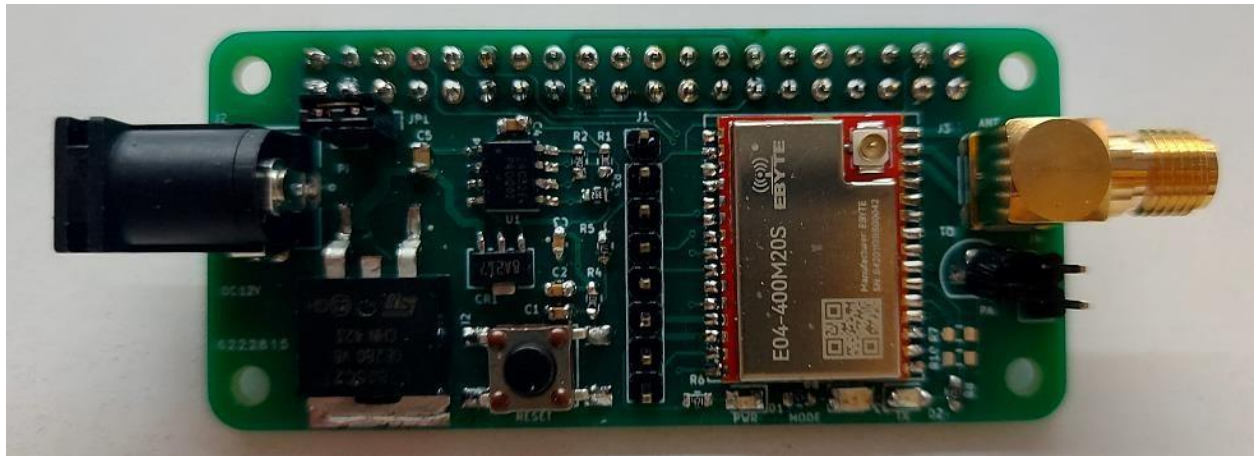


Figure 8 Pi Zero HAT Mini-Node

Updating to a Revision A3 board

As of the firmware release 1.5 in January 2026, a modification is required to any board that is not revision A3 requires for it to operate properly. The modification is in four steps:

Step 1

Remove the resistor marked 'R10', as shown in Figure 3. Remove the flux first with desoldering braid, and then hold an iron on one side of the component, the heat should spread through it enough to remove it.



Figure 3 Modification Step 1

Step 2

Using a Dremel tool or knife, cut the trace on the underside of the board as shown in Figure 4.



Figure 4 Modification Step 2

Using a voltmeter, verify that there is no longer a connection between the two points shown in Figure 5.

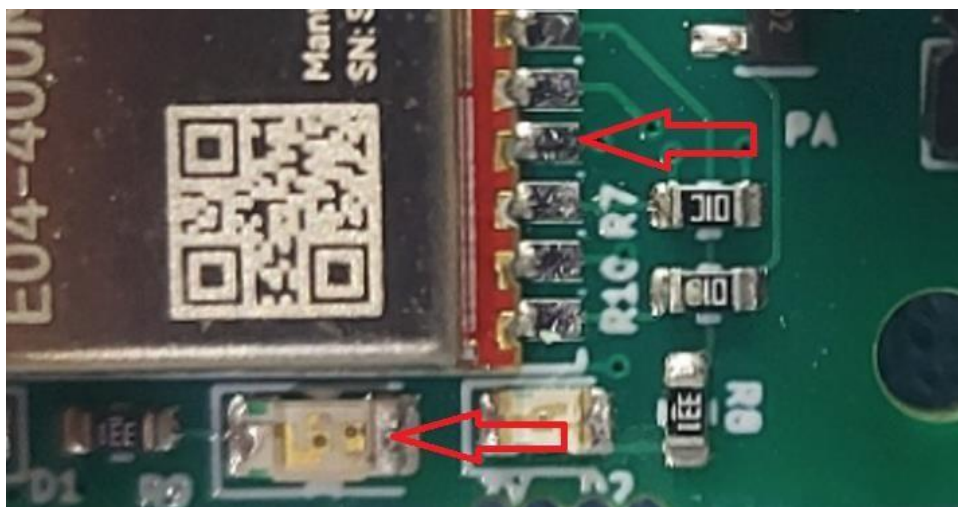


Figure 5 Verification of Step 2

Step 3

Connect a piece of number 30 Kynar wire from pin 16 of the module (bottom left) to the left hand side of the large Bi-colour LED, as shown in Figure 6. Verify this connection with a voltmeter.



Figure 6 Modification Step 3

Step 4

Connect a wire from pin 24 of the Pi header, to pin 19 of the module, as shown in Figure 7.



Figure 7 Modification Step 4

What this modification does

Removing R10 in step 1 disables the PTT key that was intended for an external amplifier that was connected to pin 16 of the module, this functionality is duplicated by the PTT LED, which can be used instead.

Step 2 disconnects one side of the bi-colour LED from pin 19 which is needed elsewhere. Step 3 reconnects the LED to pin 16 to restore that function, and step 4 connects pin 19 (PA9), to the SPI Chip Select on the Pi Header. This provides better synchronization between the Pi and the node when using the SPI.

Firmware Description

General Structure

The general structure of an IP400 Legacy node is illustrated in Figure 9.

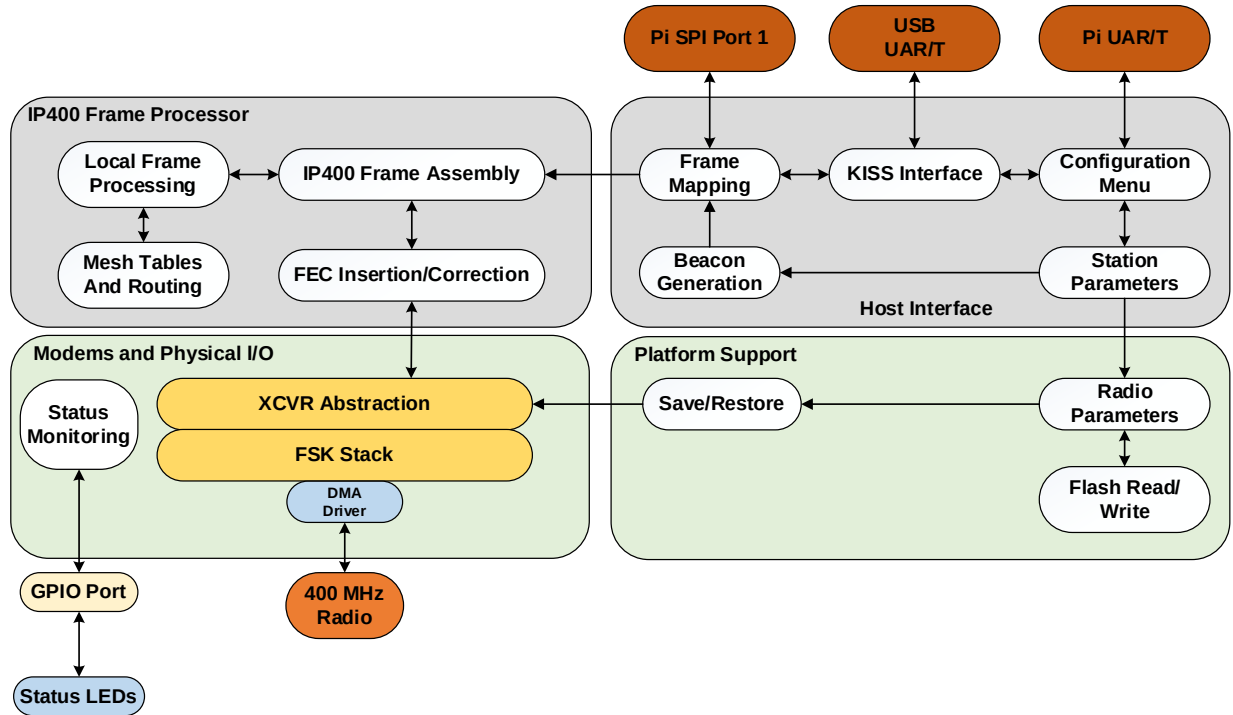


Figure 9 IP400 System General Structure

The system is structured in four sections:

1. IP400 Frame Processor. Assembles, sends and receives IP400 frames, processes mesh tables, interprets local frames, reassembles received frames and processes forward error correction.
2. Host Interface. Sends and receives frames an asynchronous serial interface in KISS format, or a Serial Peripheral Interface from another processor. Contains configuration interface and station parameters and generates beacon frames. Maps frames to IP400 format.
3. Modems and Physical I/O. Modem handlers for internal and external modems, and interfacing to FPGA co-processors, status monitoring and LED displays.
4. Platform Support. Setup and monitoring of devices such as Codecs and clock generators, and setup parameters for the radio modules, flash programming and permanent setup storage.

Software Features

Table 11 lists the firmware features that are contained in each subsystem, and the hardware platforms that support them.

Feature	System	Description	Nucleo CC2	Mini Nodes
Configuration Menu	Host Interface	An intuitive menu to set up parameters such as frequency, mode, power, callsign, etc.	✓	✓
Beaconing		A station identification message sent periodically to all stations	✓	✓
AX.25 compatibility and KISS mode		A serial interface that accepts and generates AX.25 frames in KISS format	✓	✓
Serial Peripheral Interface (SPI)		A high speed serial interface to a host processor that accepts IP frames		✓
Mesh table	IP400 Frame Processor	A collection of all stations heard and routing information to find them.	✓	✓
Chat mode		A simple chat application for sending messages between stations.	✓	✓
Ping mode		A dedicated message to request a response from a remote station. Manual or timed.	✓	✓
Repeat (Digipeater)		An ability to repeat frames based on station callsigns.	✓	✓
Real Time Clock	Platform Support	A real time clock with accuracy of 5 seconds to timestamp mesh entries.	✓	✓
Status and LED display	Modems And I/O	A series of LEDs to indicate the node status, which can be on/off or flashing.	✓	✓
400 MHz FSK		4FSK 50 Ks/sec data mode	✓	✓

Table 10 Firmware Features and Module

Table 11 lists the firmware features by subsystem.

Sub System	Feature	Description	Power Node	AMNC
Host Interface	Configuration Menu	An intuitive menu to set up parameters such as frequency, mode, power, callsign, etc.	✓	✓
	Beaconing	A station identification message sent periodically to all stations	✓	✓
	AX.25 KISS mode	A serial interface that accepts and AX.25 frames in KISS format	✓	✓
	Serial Peripheral Interface (SPI)	A high speed serial interface to a host processor that accepts IP frames	✓	✓
Ethernet	Telnet Server	Menu and Kiss interface access	✓	
	TFPT Server	Firmware updates	✓	
	UDP Server	IP400 or AXUDP packets	✓	
	HTTP Server	Status and configuration	✓	
	Positioning	AZ/EL position from magnetometer	✓	
IP400 Frame Processor	Mesh table	A collection of all stations heard and routing information	✓	✓
	Chat mode	A simple chat application	✓	✓
	Ping mode	A dedicated message to request a response from a remote station.	✓	✓
	Repeat (Digipeater)	An ability to repeat frames based on station callsigns.	✓	✓
Platform Support	Real Time Clock	A real time clock with a resolution of 10 seconds	✓	✓
	TLV320 Codec	Dual channel audio codec and radio port	✓	✓
	Clock Generator	Accurate PLL-based clock generator for data transmission	✓	✓
Modems And I/O	Status and LED display	A series of LEDs to indicate the node status	✓	✓
	Internal Modem	OFDM-AB	✓	✓
	External Modem	Interfacing for an external modem		✓

Table 11 Firmware Features and Module

Subsystem detailed description

Host Interface

KISS Serial Interface

The KISS [4] serial interface connects directly to the serial port of the processor to provide an alternate data path for the audio band OFDM modem, using an asynchronous serial interface. The serial format is 1 start bit, 1 stop bit, 8 data bits, no parity, 1 bit/ baud, baud rate 115200 symbols/second. KISS defines four special characters in its alphabet for framing purposes, as listed in Table 12 and the general form of a KISS frame is illustrated in Table 13. C notation is used to denote hexadecimal values. Both the Pi and USB Serial interfaces support the KISS mode but are mutually exclusive.

Character	Hex Code	Purpose
FEND	0xC0	Delimits the beginning and end of a frame
FESC	0xDB	Escape, indicates the subsequent character is an escape code
TFEND	0xDC	Transposed (escaped) frame end
TFESC	0xDD	Transposed frame escape

Table 12 Kiss framing characters

Begin	Command	Data 0...n	End
FEND	High Nibble – Port Index Low Nibble - Command	Data	FEND

Table 13 KISS frame format

KISS Commands

The supported command types are listed in Table 14.

Command Bytes	Frame Type	Argument Length	Description
00 [...]	Data frame	1-1024	Sends a data frame
01	Tx Delay	1	Not implemented, reserved for future use
02	Persistence	1	
03	Slot Time	1	
04	Tx Tail	1	
05	Duplex	1	
06 P S	Set hardware	2	Set power output and Squelch
07	Set Configuration		Reserved for future use
08	Get Version	0	Responds with firmware version number
09 T	Test mode	1	Initiates an internal test mode
0A	Set Serial number	0	Reserved for future use
0B	Get all parameters	0	Displays critical parameters
0C	Ack Mode		TBD
0D	Reserved	0	Reserved for future use
0E	Get serial number	0	Returns the 16-byte serial number

Table 14 Kiss Commands.

Send Data Frame (00)

All data frames are sent in AX.25 format. In the event that a data packet contains the FEND or FESC characters, the transposed mode is used instead. Figure 10 illustrates an example where the FEND (0xC0) and FESC (0xDB) are included as data bytes. Figure 10 illustrates the format of a data frame, and the escape sequence for sending framing characters as data. The data portion cannot exceed the maximum IP400 payload size of 1024 bytes.

Send the bytes 0xC0, 0xDB out of TNC port 0

C0 - FEND	00 - DATA FRAME: port 0	DB - FESC	DC - TFEND	DB - FESC	DD - TFESC	C0 - FEND
-----------	-------------------------	-----------	------------	-----------	------------	-----------

Figure 10 Use of transposition in KISS frames

Set Transmit Delay (01)

Sets the transmitter key up delay in 10 ms units. The default start-up value is 50 (i.e., 500 ms).

Set Persistence (02)

Sets the persistence parameter, p, scaled to the range 0 - 255 with the following formula, $P = p * 256 - 1$. The default value is $P = 63$ (i.e., $p = 0.25$).

Set Slot Time (03)

Sets the slot interval in 10 ms units. The default is 10 (i.e., 100ms).

Set Transmit Tail (04)

Sets the time to hold up the TX after the Frame has been sent, in 10 ms units. This command is obsolete and is included here only for compatibility with some existing implementations.

Set Duplex (05)

Sets the duplex mode, 0 for half duplex, nonzero for full duplex. Default is half duplex.

Set Hardware Command (06)

These commands are specific to a particular hardware implementation.

Set Test Mode (08)

Three different test modes are implemented as listed in Table 15.

Mode	Test
0	Turns all test modes off
1	Initiates an unmodulated CW signal at the transmitter
2	Sends a fixed tone of 1500Hz.

Table 15 Test modes

Set ACK Mode (0C)

TBD.

Reserved (0D)

This mode is unused and reserved for future development.

Frame Mapping

The mapping of AX25 to IP400 frames is a process that provides a transparent and lossless translation from one format to the other. According to the Tucson Amateur Packet Radio Group [5], an AX25 data frame is made up of three fields, as illustrated in Table 16.

Address	Control	Payload
14 to 28 bytes	2-3 bytes	0-1024 bytes

Table 16 AX25 Frame Format

The address field is mapped into an IP400 MAC header, the control and payload fields are send verbatim.

Address Field Format

The address field consists of two sub-fields, the first field is 14 bytes in length for peer-to-peer communications, the second subfield contains up to two repeater addresses, which can be 0, 7, or 14 bytes in length. Table 17 illustrates the header fields.

Byte Index	Contents	Format/Field Contents	Required/Optional
0-6	Destination Callsign	ASCII value AAAAAAA0	Required Fields
7	Destination SSID	CRRSSID0	
8-13	Source Callsign	ASCII value AAAAAAA0	
14	Source SSID	CRRSSIDE	
15-20	Repeater 1 Callsign	ASCII value AAAAAAA0	Optional
21	Repeater 1 SSID	HRRSSIDE	
22-27	Repeater 2 Callsign	ASCII value AAAAAAA0	Optional
28	Repeater 2 SSID	HRRSSIDE	

Table 17 AX.25 Address field format

The fields are illustrated in Table 18.

Field Designation	Contents
AAAAAAA	7-bit subset of the ascii character code, from 00 to 7F ₁₆
C	Command/Response bit. See Table 19.
RR	Reserved bits. Set to 1
SSID	4-bit SSID address
E	Address extension bit. Set to 1 to indicate the end of the field.
H	Has been repeated. Set to one only when the frame has been repeated.

Table 18 Address fields coding

The Command/Response bit coding identifies the frame as a command or response. The bit coding is illustrated in Table 19.

Frame Type	Destination SSID C-Bit	Source SSID C-Bit
Command	1	0
Response	0	1

Table 19 Command/Response bit coding

Mapping Rules

The following are the mapping rules for AX.25 frames:

1. The callsign fields are compressed into four-bytes using excess-40 notation.
2. The SSID, H and C bits are contained in the VPN address, as shown in Table 32.
3. The MAC addresses are made up from the source and destination addresses.
4. The repeater addresses, if present, are coded into the hop table.
5. The flag fields are set as follows:
 - a. Hop Count, 1 or 2 depending on source frame.
 - b. Coding, constant '0110', for an encapsulated AX.25 frame.
 - c. SSID – constant 0.
 - d. AX.25/VPN constant '1'.
 - e. Source call, dest call and repeat bits, all set to zero.

All data from the end of the address field to the end of the frame is copied verbatim to the payload portion of the IP400 frame, after the hop table, if present.

Serial Peripheral Interface

One of the FPGA based SPI ports implements a 16 bit slave to provide a digital audio port, in PCM-16 format, to and from the raspberry pi. The sample rate is converted internally to and from 8 KHz using decimation and interpolation filters. The PTT control and COR status can also be read in these registers.

SPI Header Field

Table 20 compares the previous SPI header format to the revised format. The previous 2 bytes for the offset are no longer used, and the hop count has been replaced with the MSB of the flag field. The maximum payload size remains at 500 bytes, inclusive of callsign extensions and hop table fields.

Header Field	Previous Format	New Format	Size (bytes)
Eye	IP4C		4
Status	Status		1
Offset Hi	High Byte of offset		2
Offset Lo	Lo Byte of offset		
Length Hi	Hi Byte of length		2
Length Lo	Lo Byte of length		
From Call	From Call		4
From VPN	From VPN		2
To Call	To Call		4
To VPN	To VPN		2
Coding	Coding		1
hop Count	Hop Count	Flags MSB	1
Flags	Frame flags	Flags LSB	1
Size in bytes			24

Table 20 SPI Frame Header format

Table 21 identifies the flag fields in the header. The MSB byte contains information about the packet coding and target modem, the LSB byte has channel coding information as well as callsign extensions.

Frame Flags	Size (bits)	Purpose	IP400 Frame field	SPI Header field
Destination Modem	2		Flags MSB	Status Byte
FEC Method	3			Coding byte
Constellation Type	3		Flags LSB	Flags MSB
Bandwidth	2			
Vacant	2			
Hoptable	1			
srcExt	1			Flags LSB
destExt	1			
payloadMSB	1			

Table 21 Frame flags definition

Table 22 defines the fragmentation field values.

Value	Type	Length	Payload content
0	Self contained	<500	Complete frame
1	First fragment	500	First 500 bytes of fragmented frame
2	Middle fragment	500-999	Second or last fragment, up to 500 bytes
3	Last fragment	1000-1499	Final fragment

Table 22 Fragmentation field values

Table 23 defines the packet coding values.

Value	Type	Purpose
0	UTF8_TEXT_PACKET	Used by internal chat application
1	DATA_PACKET	Download data for firmware upgrade
2	BEACON_PACKET	Beacon packet
3	AX_25_PACKET	AX.25 encapsulated packet
4	IP_ENCAPSULATED	TCP/UDP encapsulated packet
5	ECHO_REQUEST	Echo request to remote station
6	ECHO_RESPONSE	Echo response from remote station
7	LOCAL_COMMAND	Request for routing table lookup
8-15	USER_DEFINED	Available for user use

Table 23 Packet coding

Table 24 lists the modem qualifier fields and meaning.

Field	00	01	10	11
Modem	4FSK Primary	4FSK Secondary	OFDM-AB	Reserved
FEC Method	None	CCITT CRC-16	Trellis Coding	Reserved
Bandwidth	NB	ENB	WB	EWB

Table 24 Modem qualifier fields

SPI to IP400 Header mapping

Table 25 illustrates how SPI headers are mapped into an IP400 Frame header.

SPI Source Data			Destination	
Field	Subfield	Size	Frame Header	Size
Header (23 bytes)	Eye	4	n/a	
	splStat	Lower 4 bits	Flags: fragmentation	2 bits
	Offset_hi	1	n/a	
	Offset_lo	1	n/a	
	Length_hi	1	Flags: payloadMSB	1 bit
	Length_lo	1	Length	1 byte
	From Call	4	Source MAC [0..3]	4 bytes
	From VPN	2	Source MAC [4..5]	2 bytes
	To Call	4	Dest MAC [0..3]	4 bytes
	To VPN	2	Dest MAC [4..5]	2 bytes
	Coding	4 bits	Flags: coding	4 bits
	Hopcount	1 byte	n/a	
Flags (1 byte)	Fragmentation	2 bits	Flags: fragmentation	2 bits
	Hoptable	1 bit	Flags: hoptable	1 bit
	SrcExt	1 bit	Flags: srcExt	1 bit
	DestExt	1 bit	Flags: destExt	1 bit
	Command	1 bit	n/a	
	noConnect	1 bit	n/a	
	Repeat	1 bit	n/a	
Ext Src Call	Callsign	4 bytes	srcExt	4 bytes
Ext Dest Call	Callsign	4 bytes	destExt	4 bytes
Hop Table	Address (2)	6-12 bytes	MAC [0..5]	6-12 bytes
(14 bytes)	Flags(2)	2 bytes	MAC [0..5]	2 bytes
Payload	Data	500-(Ext Src)-(Ext Dest)-(Hop Table)		

Table 25 SPI to IP400 Header Mapping

IP400 Frame Processor

Figure 11 illustrates the frame processor module from the generic structure diagram.

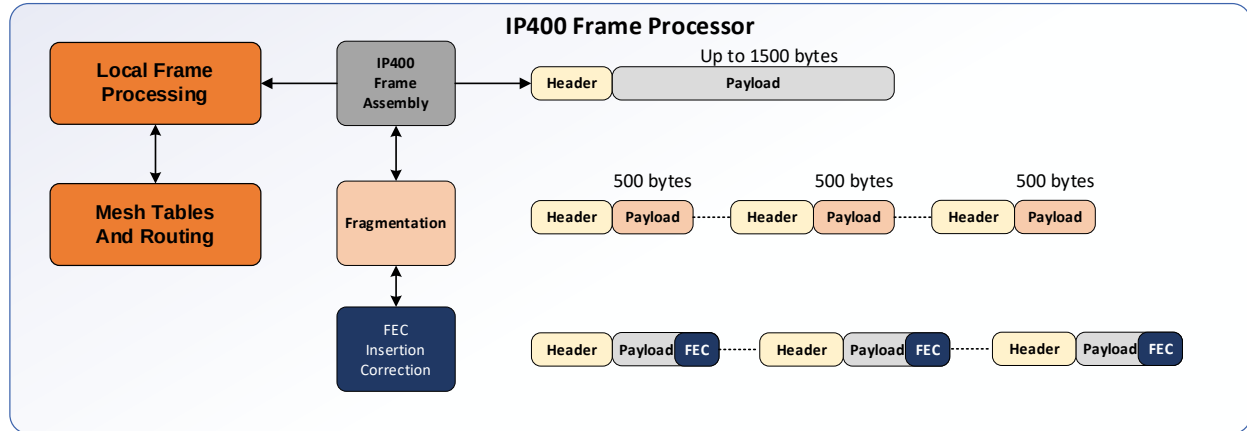


Figure 11 IP400 Frame Processor

Local Frame Processing

Table 26 illustrates the packet coding and where they are routed for processing. There are 16 packet types defined, the first 8 are processed internally, the remaining types are available for user applications, which are routed to the SPI interface by default.

AX.25 frames can only be originated at the KISS interface and are routed back to it on receive. Cross-modes between KISS and SPI are currently not supported and may produce unpredictable results.

The command frame is reserved for future use as a 'DNS lookup' mode. The intent is to be able to find the routing information to a specific node by using this frame type. Implementation is still pending.

Packet Type	Coding	Purpose	Application/Destination
UTF8_TEXT_PACKET	0	Text packet	Chat
DATA_PACKET	1	Data packet	VPN Specific; KISS or SPI
BEACON_PACKET	2	Mesh Discovery	Mesh tables
AX_25_PACKET	3	Encapsulated AX.25 packet	KISS interface
IP_ENCAPSULATED	4	Encapsulated IP packet	SPI Interface
ECHO_REQUEST	5	Request for a response	Chat
ECHO_RESPONSE	6	Response to a request	Chat
LOCAL_COMMAND	7	Routing enquiry	Mesh Tables
USER_DEFINED	8-15	Undefined	SPI Interface

Table 26 Local Frame Processing

Mesh Tables and Routing

Field	Data Type	Source
Status	MeshTableStatus	Beacon frame, TOD clock
macAddr	IP400 MAC address	Beacon frame
NextSeq	Integer	Last frame
lastRSSI	Integer	Receiver
txPower	Integer	Beacon Frame
Capabilities	SETUP_FLAGS	Station capabilities
LastHeard	TIMEOFDAY	Last time heard
hopCount	Integer	Number of hops required
RptAddr	IP400 MAC address	Repeater addresses (max 2)

Table 27 Mesh Table Contents

Frame Assembly

Each IP400 frame contains a header and potentially a hop table which are the same size. There is no forward error correction, just a CRC-16 checksum to verify validity. The fields in the header are listed in Table 28. Callsigns are compressed using a radix-40 scheme.

Field	Size (bytes)	Purpose in Header Symbol	Purpose in Hop Table
Source MAC Address	6	Source MAC address	Repeater address 1
Destination MAC Address	6	Destination MAC address	Repeater address 2
Flags	2	Frame flags	Repeat Flags
Payload length LSB	1	Number of payload packets	Not Used
Sequence number	4	Packet sequence number	
Source address extension	4	Last 6 bytes of source callsign	
Dest. address extension	4	Last 6 bytes of destination callsign	
CRC	2	CCITT CRC-16 Checksum	
Total Bytes	29		

Table 28 Header Symbol Coding

The flag field contents for the headers symbol are listed in Table 29.

Bits	Field	Purpose
15-14	Fragmentation	Fragmentation status
13-10	Data Type	Packet data type
9-8	Bandwidth	On air occupied bandwidth
7	FEC Present	Indicates FEC is present
6-4	Constellation	Constellation used
3	Hop table	Hop table present
2	Source Extended	Source address uses extension field
1	Destination Extended	Destination address uses extension field
0	Payload length MSB	MSB of payload length (Type 0 and 3 only)

Table 29 Frame Header Flag field definitions

The flag field contents for the hop table are listed in Table 30.

Bits	Field	Purpose
15	Rpt 1 valid	Repeater address 1 is valid
14	Rpt 1 hbr	Repeater 1 'Has been repeated' flag
13-8	Unused	
7	Rpt 2 valid	Repeater address 2 is valid
6	Rpt 2 hbr	Repeater 2 'Has been repeated' flag
5-0	Unused	

Table 30 Hop Table Flag field definitions

Table 29 lists the value of the multi-bit fields.

Field Name	Value	Meaning
Fragmentation	0	Self-contained frame
	1	First frame of a segmented frame
	2	Middle frame
	3	Last frame
Bandwidth	0	NB
	1	ENB
	2	WB
	3	EWB
Constellation	0	BPSK/2FSK
	1	QPSK/4FSK
	2	8QPSK/2GFSK
	3	QAM-16/4GFSK
	4	QAM-32
	5	QAM-64
	6	QAM-128
	7	QAM-256

Table 31 Flag field values

MAC Address Coding

The MAC addresses is composed of the first 6 bytes of the callsign compressed into 4 bytes, followed by a unique station identifier (USI).

Table 32 illustrates the four methods of coding a station address. There are two broadcast address types, all 1's in the address is recognized by all nodes on a network, an 'All call' address contains the callsign only with the remaining fields set to all 1's and is recognized by all nodes with the same callsign, regardless of the USI.

There are two methods to uniquely address a node, by VPN or AX25 address. In the VPN mode, the USI is a 2-byte field composed from a unique identifier register within the node hardware, and in the AX25 compatibility mode the 5th byte is set to all 1's and the last byte contains the secondary station identifier (SSID), and 'C' and 'H' bits from the AX.25 protocol (denoted X). Table 32 illustrates the VPN coding.

Type	Callsign Field				Unique Station Identifier	
Byte	1	2	3	4	5	6
Broadcast	FF ₁₆					
All call	Callsign 1-6 in Excess 40 format				FF ₁₆	
VPN Address					VPN Address	
AX25 Compatibility					FF ₁₆	101XSSSS ₂

Table 32 Station address coding

Callsign Compression

The callsign field characters are limited to the code set as illustrated in Table 33.

Character	Coding	Character	Coding	Character	Coding	Character	Coding
0	0	Space	10	J	20	T	30
1	1	A	11	K	21	U	31
2	2	B	12	L	22	V	32
3	3	C	13	M	23	W	33
4	4	D	14	N	24	X	34
5	5	E	15	O	25	Y	35
6	6	F	16	P	26	Z	36
7	7	G	17	Q	27	_	37
8	8	H	18	R	28	-	38
9	9	I	19	S	29	@	39

Table 33 Callsign character coding

The field shall be encoded using the following formula. Unused characters shall be replaced with a space (00).

$$F(3,0) = \sum_{n=1}^{n=6} C(n) + C(n-1) \times 40$$

Where F is the field value in bytes, C is the character and n the index into the callsign field. Each callsign can be up to 6 characters. The maximum value of a legally encoded callsign field is F423FFFF. A value of all 1's (FFFFFFF) is a broadcast address to all stations, addresses outside these limits are not used.

Hop Table

The Hop table contains up to 2 MAC entries composed of a duality of inbound and outbound routing information. For inbound packets that have a different or broadcast destination field and are repeatable, the repeating station adds it callsign on the end and repeats the frame. For outbound packets, it contains routing data to the destination. If the packet is not destined for a receiving station, and its callsign is at the end of the table, it is removed and repeated.

Fragmentation

The fragmentation scheme limits the payload data size to 500 bytes, which enables a full-size UDP or TCP frame to be transmitted in three successive symbols. Each symbol contains the same header information, but a field in the header identifies the frame as one of three following types:

1. Single frame. A self-contained frame with a payload of less than 500 bytes.
2. Start frame. The first of a series of frames containing the first 500 payload bytes.
3. Middle frame. The second of a series of frames containing bytes 501 to 1000 of the payload.
4. Final frame. **Either** the last in a two-frame series containing up to the last 500 bytes, **or** the last in a series of three, containing bytes 1001 onward to a maximum of 1500.

Figure 12 illustrates the fragmentation scheme.

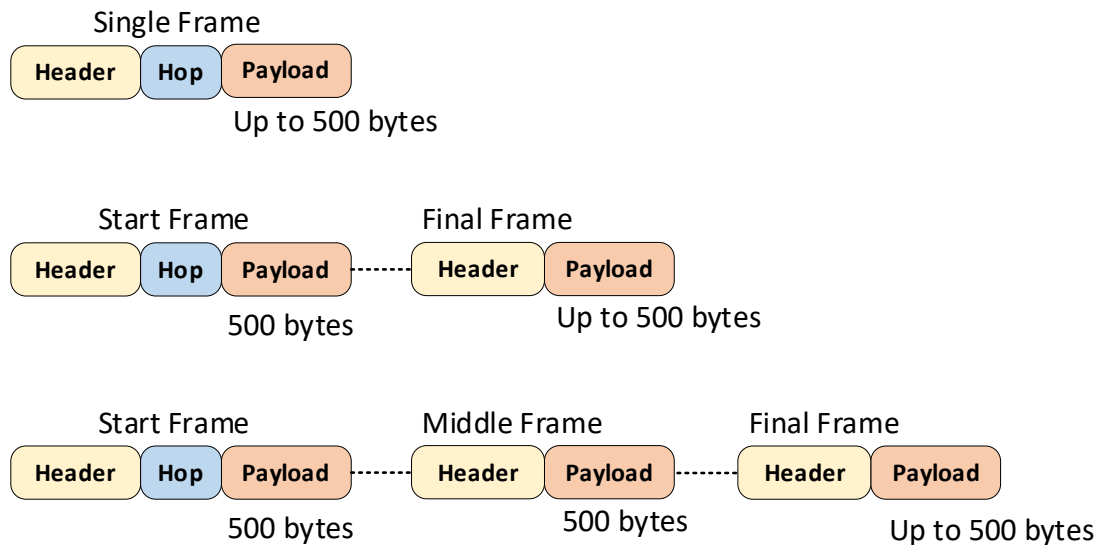


Figure 12 Frame Fragmentation Schemes

Platform Support

This system varies between different node types. The minimum functionality implemented contains radio specific parameters for each radio type, which are modified in the configuration menu.

Radio Parameters

Table 34 illustrates the radio setup parameters.

Parameter	Purpose	Value	Purpose
Mode	Specifies data mode	0-3	See Table 35.
Symbol Coding	Carrier Coding Scheme	0-7	See Table 36.
Frequency	Operating Frequency	420.0-449.95	Operating frequency ³

Table 34 Radio Parameters

Table 35 illustrates the mode values.

Field Value	Standard
0	50Ks/Sec
1	Unused
2	
3	

Table 35 Data mod values for OFDM-AB standards

Table 36 defines the symbol coding.

Field Value	Bits	Constellation
0	1	BPSK
1	2	4FSK
2-7		Unused

Table 36 Data Symbols size definitions

³ Used on the mini node only, and the lower 10MHz (420-430) is not available in all countries

Modems and Physical I/O

This system contains the software support for each of the modem types, including any signal processing required for software defined modems.

Transceiver Architecture

Figure 13 illustrates the architecture of the 4FSK modem and stack.

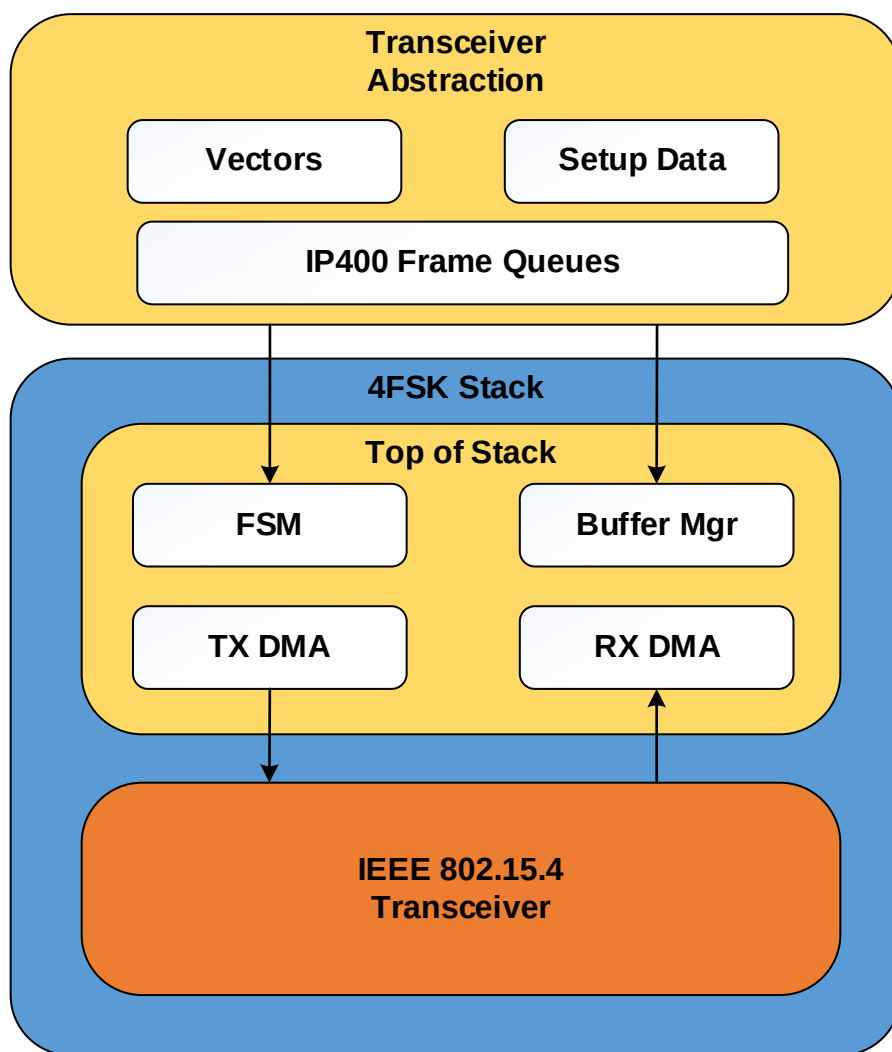


Figure 13 4FSK Transceiver Stack

Transceiver Abstraction

At the top is a module that enables different transceivers to be supported by the firmware. This module contains the default setup data for the modem type, critical vectors to access processing methods within each transceiver module, and queues for transmit and receive frames.

The Mini Node and Nucleo modules only supports a single modem which conforms to the IEEE 802.15.4 standard and is built into the CPU hardware.

Stack

The 4FSK Stack consists of two layers:

1. **Top of Stack.** This module contains a state machine to manage the receiver and transmitter, and the transitions. It also contains a buffer manager that converts IP400 frames into a series of bytes, and vice versa, thus decoupling the internal frame architecture from the modem. The transmit module in this layer is responsible for converting bytes into symbols and creating the sync symbol and a CRC, and the receive module contains a mini state machine to look for synchronization, decode symbols and verifying the CRC.
2. **IEEE 802.15.4 Modem.** The modem is contained in the CPU hardware, and uses DMA transfers for transmit and receive.

Status Monitoring

Table 37 illustrates the usage of the three LEDs on the hardware platforms:

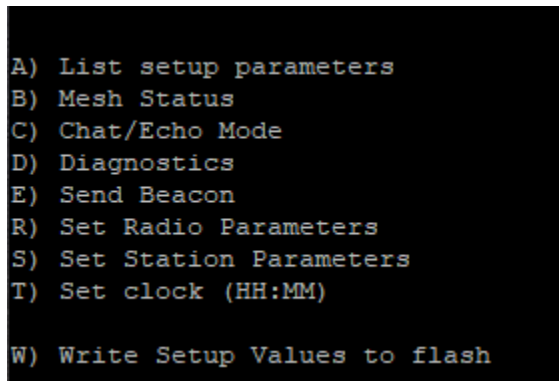
LED	Usage	Colour	Indication
PWR	Power on	RED	Power is on
MODE	Rx	GREEN	Receiver is active
	Error	RED	An error has occurred
TX	Transmit	RED	Transmitter is on

Table 37 LED Usage by platform

Operating the Software

Main Menu

The main menu is invoked by connecting to the UAR/T on the node with a serial connection at 115200bps. After connecting, the enter key will invoke the menu. It is illustrated in Figure 14.



```
A) List setup parameters
B) Mesh Status
C) Chat/Echo Mode
D) Diagnostics
E) Send Beacon
R) Set Radio Parameters
S) Set Station Parameters
T) Set clock (HH:MM)
W) Write Setup Values to flash
```

Figure 14 Main Menu

The menu items are described in Table 38.

Menu Item	Function
A	Lists the current setup parameters, including clock, station and radio
B	Displays the mesh table contents
C	Enters chat mode and echo modes
D	Enters the diagnostics menu
E	Sends a beacon frame immediately
R	Sets the radio parameters
S	Sets the station parameters
T	Sets the time of day clock
W	Writes the setup data to the flash

Table 38 Main Menu Items

Item W is used to write the setup parameters to the flash memory after modification. When powering up, if the memory is valid, the parameters are loaded. If not, then a default set of parameters is used, which can be modified accordingly.

It is possible that reloading the firmware may cause the parameters to revert to the default. This is due to changes in the setup memory layout.

Setup Parameters

A list of all the current setup parameters can be seen from the main menu item 'A'. There are four groups of parameters:

1. Firmware version, system time, radio ID and VPN address.
2. Station callsign, AX.25 compatibility mode and description
3. Location data and capabilities
4. Repeat (digipeat) mode enable
5. Beacon interval

Figure 15 illustrates them.

```
Firmware version: 2.0, Revision: 125 (Beta)
Build Date: 2026-04-10 19:24:12 -0600 (Fri, 10 Apr 2026)
System time is 00:02:50
Radio ID is 304c1389938c347a
VPN Address 172.29.39.243

Station Callsign->VE6VH
AX.25 and KISS Mode Not Enabled
Description->Lab Nucleo Board

Latitude->51.143
Longitude->-114.176
Grid Square->DO21vd
Capabilities->FSK KISS
Repeat mode on
Beacon Interval->5 mins
```

Figure 15 Setup Parameters

Mesh Table

Figure 16 illustrates the mesh table contents, and Table 39 explains them.

```

Nodes Heard: 2
Call      VPN Addr  Status  RSSI    Seq     Last Heard  Hops  Capabilities
VE6VH     172.29.222.1  OK      -8      11040   15:06:10   0     FSK RPT 14 dBm
  
```

Figure 16 Mesh Table

Item	Explanation
Call	Callsign of the transmitting station. In AX.25 mode, also has the SSID
VPN Address	The network VPN address of the station
Status	OK if heard in the last hour, LOST if not.
RSSI	Receive signal strength when SYNC received (in dBm)
Next Seq	Next anticipated sequence number
Last Heard	TOD clock reading when last heard
Hops	Number of repeat hops, 0 indicates a direct signal
Capabilities	Data rate, repeat capabilities and transmitted signal strength

Table 39 Mesh Table explanation

The station capabilities can only be populated upon receipt of a beacon frame. If a non-beacon is received from a new station, it is added to the table without this field. If a beacon is heard subsequently, all the fields are updated. See Table 40 for details of these.

Bit	Capability	Meaning
0	FSK	Station can send 2 or 4 FSK
1	OFDM	Station is capable of OFDM transmissions
2	AX.25	Station has the AX.25 mode enabled
3	REPEAT	Station can act as a digipeater
4-7	EXT	SSID when in AX.25 compatibility mode

Table 40 Capabilities field

Chat/Echo Mode

The chat mode is an interactive exchange between two nodes. Frames can be broadcast to all nodes, to all nodes with the same callsign, or to a specific node.

When invoked, it defaults to the chat mode with a broadcast address. Control keys are used to change the chat status, as shown in Table 41.

Key	Purpose
ESC	Enables entering a new destination address.
CTRL+'R'	Toggles repeat mode
CTRL+'E'	Enters echo mode
CTRL+'D'	Toggles dump mode
CTRL+'Z'	Exits chat and returns to the main menu
ENTER	Sends the current frame/ends an entry
BACKSPACE	Deletes the last character entered, or more if repeated

Table 41 Chat mode control keys

Sending a Chat Frame

First destination address must be set, by depressing the ESC key when the cursor is at the beginning of the line, from that point there are three options:

1. Depressing the ENTER key right away will set the address to broadcast, which reaches all nodes on the network.
2. Enter a callsign and depress the ENTER key. From there one of three things can happen:
 - a. If the call is not in the mesh table, it will be rejected, and the last destination will remain unchanged.
 - b. If that call is unique and, in the table, then the destination will be set to the MAC address of that node. Frames will only be sent to that particular one.
 - c. If that call is not unique, the destination will be set initially to all nodes with that callsign, and a list of all the entries found is displayed. Each entry has a callsign followed by a dash ('-') and an index into the mesh table.
3. Enter callsign, dash and index. If found in the table, the destination will be set to that node only.

Chat frames are displayed as follows:

- Originator callsign and VPN address
- Destination callsign and VPN address
- Number of times the frame was repeated
- Sequence number of the frame
- Data message

Dump Mode

The dump mode changes the display format to hexadecimal and also includes the frame header information. Useful for debugging.

Echo Mode

The echo mode is designed to query a node to see if it can be reached, and whether it will respond. As with the chat mode, the destination address must be set first. The CTRL/E key is multi-functional as follows:

1. The first depression enters the interactive mode.
2. The second enters a timed mode.
3. The third returns to normal chat mode.

In the interactive mode. Depressing the 'Enter' key will send a query to the remote station. The response is displayed in the current chat mode.

In the timed mode, an echo request frame is sent to the remote station approximately once every 5 seconds.

Diagnostics Menu

Figure 17 illustrates the diagnostics menu and Figure 17 explains them.

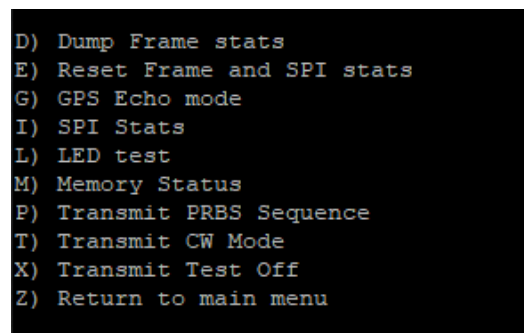


Figure 17 Diagnostics Menu

Menu Item	Function
D	Dumps the frame statistics
E	Reset all statistics
G	Enables GPS Echo mode, if installed
I	Dumps the SPI statistics
L	Runs an LED test
M	Displays the memory status
P	Transmits a constant PRBS sequence
T	Transmits a CW signal
X	Turns of previous tests and returns to normal operation
Z	Returns to the main menu

Table 42 Diagnostics Menu

In the OFDM-AB transceiver the test modes are replace with:

P	Transmits a 1KHz tone
T	Turns on the transmitter and loops receive audio to transmit

Frame Statistics

Statistics are kept at the modem and frame processor levels. Each modem has its own set of statistics, particular to that modem type, but the frame processor is common to all modes.

WL33 Modem Statistics

Figure 18 illustrates the statistics for the WL33 modem.

```

Statistics for Radio port 1: 'WL33'
  FSM State: 16: Receive
  Code FSM State: RX_ACTIVE
  Transmitted frames->2
  Received frames->1
  CRC Errors->33
  Rx Timeouts->0
  Invalid IP400 Frames->0
  Rx Dequeued Frames->1
Xmit Buffer Stats
  State: EMPTY
  Timer State: NOT RUNNING (0)
  Frames Buffered->2
  Buffers Xmitted->2
  Tx Buffer size->2048
  Tx Buffer available->2048
  
```

Figure 18 WL33 Modem Statistics

Item	Explanation
FSM State	See manufacturers' data sheet [2]
Code FSM State	State of the internal code state machine
Transmitted Frames	The number of transmitted frames since the last restart
Received Frames	Frames received with good CRC
CRC Errors	The number of frames received with FEC errors.
Rx Timeouts	The number of times the receiver timed out
Invalid IP400 Frames	Frames with an unrecognizable header
Rx Dequeued frames	Frames sent on for internal processing
Xmit Buffer stats	Status of the transmit buffer
State	Current state of the buffer
Timer State	State of the countdown timer
Frames Buffered	Number of frames that were queued in the buffer
Tx Buffer Size	Size of the buffer
Tx Buffer available	Number of bytes available in the transmit buffer

Table 43 Frame Statistics

Frame Processor Statistics

Figure 19 illustrates the frame processor statistics.

```
Frame Statistics
Frames processed->1
Duplicate frames->0
Beacon frames->1
Chat frames->0
Kiss frames->0
Undecoded frames->0
Echo Request frames->0
Echo Reponse frames->0
Repeated frames->0
Unknown frames->0
Frames with my Callsign->0
Rejected Frames->0
```

Figure 19 Frame Processor Statistics

Table 44 illustrates the frame processor statistics

Item	Explanation
Frames Processed	Number of frames presented to the frame processor
Duplicated Frames	Frames rejected as being duplicate
Beacon Frames	Beacon frames processed
Chat Frames	Chat frames processed
KISS frames	KISS frames processed
Undecoded Frames	Frames not decoded by the node
Echo Request Frames	Echo Requests received
Echo Response frames	Echo responses sent
Repeated Frames	A count of the number of frames repeated
Unknown Frames	Frames that could not be recognized
Frames with my Callsign	Frames in which my callsign was found (in digipeat mode)

Table 44 Frame Processor Statistics

SPI Statistics

Table 45 illustrates the SPI statistics

Item	Explanation
SPI State	State of the SPI: active or not active
IP400->SPI Frames	Received frames sent to the SPI
SPI->IP400 Frames	Transmit frames received from the SPI
SPI Single Frames	Frames with a payload less than 500 bytes
First Fragment Frames	First 500 bytes of a larger frame
Last Fragment Frames	Remainder of a large frame
Discarded Frames	Frames without the correct 'IP4C' eye

Table 45 SPI Statistics

Memory Status

This menu item is for developers to determine if any memory leaks have occurred. Refer to the source code for calls to the allocator, and the Free RTOS manual for a description of the heap.

Send Beacon

This menu item sends a beacon frame immediately. The payload is a comma-delimited string, the contents are shown in

Field	Contents
Fix Source	Source of lat/long data, FIX for setup data, GPS from a receiver
Latitude	Current Latitude if GPS is connected, else supplied data.
Longitude	Current Longitude, same as above
Speed	Speed if in motion, null field if in FIX mode
Time	System or GPS time
Grid Square	Grid square from setup data
Description	System description from the setup data

Table 46 Beacon Frame Payload

LED Test

The LED test will cycle through the patterns shown in the following table:

Test	LED	Colour
1	LED 1	Green
2	LED 2	Green
3	LED 3	Green
4	LED 1	Red
5	LED 2	Red

Table 47 LED Test Cycling

GPS Echo Mode

The NMEA sentences from the GPS receiver are dumped to the screen as shown below:

```
GNVTG,160.14,T,,M,0.23,N,0.42,K,A*  
GNGGA,040907.000,5108.6054,N,11410.5759,W,1,07,1.53,1275.4,M,-17.5,M,,*  
GLGSA,A,3,86,85,71,,,,,,,,,1.83,1.53,0.99*  
GPGSV,4,1,14,18,73,294,36,29,50,156,22,23,42,220,26,15,40,130,29*  
GPGSV,4,3,14,27,07,315,,20,05,057,,10,03,225,16,07,02,007,15*  
GLGSV,3,1,11,86,75,007,33,70,53,059,,71,50,151,26,87,34,305,18*  
GLGSV,3,2,11,85,32,101,22,77,21,285,,78,14,344,,72,09,183,*  
GNRMC,040907.000,A,5108.6054,N,11410.5759,W,0.20,160.14,070225,,,A*  
GPGSA,A,3,13,18,29,05,15,,,,,,,,,1.74,1.42,0.99*  
GNRMC,040903.000,A,5108.6057,N,11410.5757,W,0.35,160.14,070225,,,A*  
GNVTG,160.14,T,,M,0.35,N,0.65,K,A*  
GLGSA,A,3,86,85,71,,,,,,,,,1.83,1.53,0.99*  
GPGSV,4,1,14,18,73,294,36,29,50,156,22,23,42,220,26,15,40,130,29*  
GPGSV,4,4,14,30,01,038,,45,,,*  
GLGSV,3,1,11,86,75,007,33,70,53,059,,71,50,151,26,87,34,305,18*  
GLGSV,3,3,11,76,07,243,,68,04,020,,69,01,019,*  
GNRMC,040907.000,A,5108.6054,N,11410.5759,W,0.20,160.14,070225,,,A*  
GPGSA,A,3,18,29,05,15,,,,,,,,,1.90,1.62,1.00*  
GNRMC,040908.000,A,5108.6053,N,11410.5759,W,0.26,160.14,070225,,,A*  
GNVTG,160.14,T,,M,0.20,N,0.36,K,A*  
GLGSA,A,3,86,85,71,,,,,,,,,1.74,1.42,0.99*  
GNVTG,160.14,T,,M,0.35,N,0.65,K,A*
```

Figure 20 GPS NMEA sentences

While this mode is active, two keys are processed:

1. Pressing the 'ESC' key will pause and prompt for the enter key to be hit before returning to the main menu. The data remains on the screen.
2. Pressing the ENTER key will clear the screen and return to the main menu.

Setting the clock

The time of day is entered as HH: MM. The clock has a granularity of 10 seconds and uses 24 hour format, and knows nothing about time zones, as it considers you are in your home zone.

Radio Setup Menus

To radio setup menus can be invoked from menu item 'R'. The menu is illustrated in *Figure 21*. Setup menus are dependent on the type of modem attached.

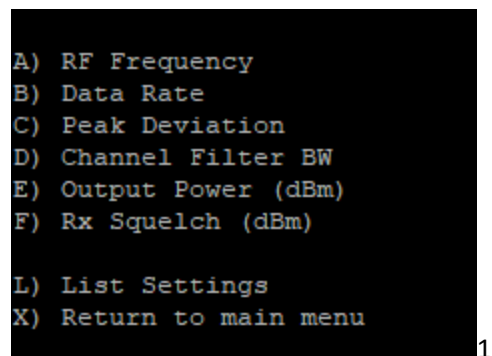


Figure 21 Radio Setup Menu

The radio setup menu items are listed in *Table 48*.

Item	Purpose	Format
RF Frequency	RF operating frequency from 420 to 450 MHz	XXX.XXX or NNNNNNNNNN
Data Rate	Data rate 1.2 to 600Kb/s	xx.xx KHz or NNNNNN
Peak Deviation	Peak FM deviation	xx.xx KHz or NNNNNN
Channel Filter BW	BW of the receiver (See <i>Table 49</i>)	xx.xx KHz or NNNNNN
Output power	Sets the transmitter power	0 to +20 dBm
Rx Squelch	Sets the receiver threshold	-30 to -130 dBm

Table 48 Setup parameters

Items are changed by invoking the relevant menu item, the 'L' key lists the current settings. The bandwidth values must be one of those listed in *Table 49*.

Permissible Bandwidth values (KHz)							
1600	666	266	100	41.6	16.6	6.1	2.6
1510	622	244	94.4	38.9	15.3	5.9	2.4
1422	577	222	88.9	36.1	13.9	5.5	2.2
1332	533	200	83.3	33.3	12.5	5.2	2.1
1244	488	188	77.8	30.5	11.8	4.8	1.9
1154	444	178	72.2	27.8	11.1	4.5	1.7
1066	400	166	66.7	25.0	10.4	4.1	2.6
976	377	155	61.1	23.6	9.7	3.8	
888	355	144	55.6	22.2	9.0	3.5	
800	333	133	50.0	20.8	8.3	3.1	
755	311	122	47.2	19.4	7.6	2.9	
711	288	111	44.4	18.1	6.9	2.7	

Table 49 Permissible Bandwidth Values (KHz)

Station Setup Menu

The station setup menu is constant for all modes and is invoked from the main menu item 'S'. The menu is illustrated in Figure 22.

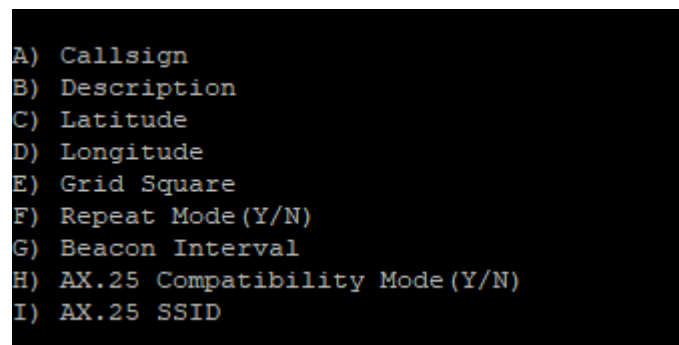


Figure 22 Station Setup Menu

Item	Purpose	Format
Callsign	Station callsign	Up to 6 characters
Description	A description of the station	Up to 32 characters
Latitude	Latitude, positive for N, negative for S	$\pm xxx.xxx^4$
Longitude	Longitude, positive for E, negative for W	$\pm xxx.xxx$
Grid Square	Home grid square	XXNNxx
Repeat Mode	Set the repeat flag in sent frames	Y or N
Beacon Interval	Interval between beacons, normally 5 mins	xx minutes
AX.25 Compatibility	Enables the AX.25 addressing mode	Y or N
AX.25 SSID	The AX.25 SSID value	0 to 15

Table 50 Station setup parameters

Note that any of these setting are not permanent until written to flash from the main menu.

⁴ Used in beacon frames, unless a GPS receiver is connected

Building the Code

All variants of the code are built using the ST Microelectronics CubeIDE platform, at version level 2.1.1 or later. There are three preliminary steps to building the code:

1. Unzip the project for the required platform and import into the IDE.
2. Import the common code.
3. Import the platform specific code.

Hardware Platform	Project zip file	Platform code	Object File	Flash Method
Nucleo CC2	NucleoCC2.zip	WL33	NucleoCC2.elf	Debugger OR
Mini Node	MiniNod.zip		MiniNode.elf	FlashNode.sh

Table 51 Code Components

Prior to building the code, add the source directories to the project first:

1. Right click on the project in the explorer window, choose 'Properties'.
2. From the dialog box, expand C/C++ General, then Paths and Symbols.
3. From the source location tab, click 'Add Folder'.
4. Choose the common source folder and click 'Apply'.
5. Repeat for the platform code as shown in Table 51.

Right click on the Project again, this time choose 'Clean Project', then 'Build Project'.

Flashing

Table 51 also lists the flashing method for each of the hardware platforms. The debugger is contained in the CubeIDE platform, otherwise the object file can be copied to the target platform and flashed with the appropriate shell script.

Those platforms that do not support a debugger must use the flash shell script method

Support Software

UDP to SPI software

The UDP to SPI software (ip400spi) has been developed to provide a mechanism to send and receive packets to the node using the IP user datagram (UDP) protocol from a raspberry pi. The software is invoked on the command line, using the parameters shown in Table 52.

Parameter	Argument	Meaning
-B	None	Sends a single test frame, built into the software in frame.c
-s	/dev/spiX.Y	Specifies the SPI port. Usually /dev/spi0.0
-n	Name or IP	Name or IP address of the host receiving the packet
-p	nnnn	Port address of the host
-m	nnnn	Sending port address
-d	x	Sets the debug level to x.
-l	n	Sets loopback mode. See below and Table 54.
-h	None	Prints the help message

Table 52 Pi utility software command parameters

The program is terminated by typing Control/C at the keyboard. It can also be run in the background, or as service.

The debug level is a bit field that has the following values:

Bit	Value	Debug
0	1	Enables logging of debug messages
1	2	Enables SPI debugging. Removes real time performance

Table 53 Pi Utility debug levels

Loopback mode

Loopback mode is a debug mode intended for debugging formatting of packets into SPI payloads. When active, the SPI is not enabled, instead a transmitted packet goes through the motions of being fragmented, but instead of being sent over the SPI, it is looped back as if it was received.

This exercises both the transmit and receive fragmentation modes, a successful test methodology should be to use the packet generator to generate packets, which are sent to a PC running Wireshark using the loopback mode. The PC should be running the dissector.

Value	
1	SPI loopback. Simulates an SPI transmit/receive cycle by copying data from Tx to Rx
2	IP400 Frame loopback. Exercises the node SPI code and loops back at frame level
All others	No loopback operation performed

Table 54 Loopback mode arguments

UDP Packet Generator

The UDP generator software (UDPGen) generates a series of packets that can be used as test cases for the UDP to SPI software. Table 55 lists the command line parameters.

Parameter	Argument	Meaning
-n	Hostname	Name or IP address of the host receiving the frame
-p	Host Port	Port number on the host
-m	Orig Port	Originating port number
-f n	Frame type	Frame type to send (see Table 56)
-r	Repeat count	Number of times to send the frame (default 1)
-D	Debug mode	Enables debug messages to be displayed
-h	Help	Prints a help message

Table 55 Command line parameters for UDP packet generator

The frame types are listed in Table 56.

Frame type	Length	Type
0	70	Beacon message
1	52	Short AX.25 encapsulated frame
2	572	Long AX.25 encapsulated frame
3	900	Huge AX.25 encapsulated frame
4	1200	Large IP encapsulated packet
5	1410	Huge IP encapsulated packet

Table 56 Generated frame types